Administration & Sécurité des Systèmes d'Exploitation UNIX

Année Universitaire
2022-2023

Chapitre II : Shell & Commandes de base

Objectifs

- Comprendre le Shell Linux et son rôle
- Manipuler quelques commandes de base
- Définir la notion des variables d'environnement
- Maîtriser le concept des liens physiques et symboliques

Plan

1. Environnement graphique de travail
2. Shell : Définition & Rôle
3. Commandes de bases
    3.1. Utilitaires d’aide
    3.2. Manipulation des fichiers et des répertoires
4. Variables d'environnement
5. Liens Physiques Vs Liens Symboliques

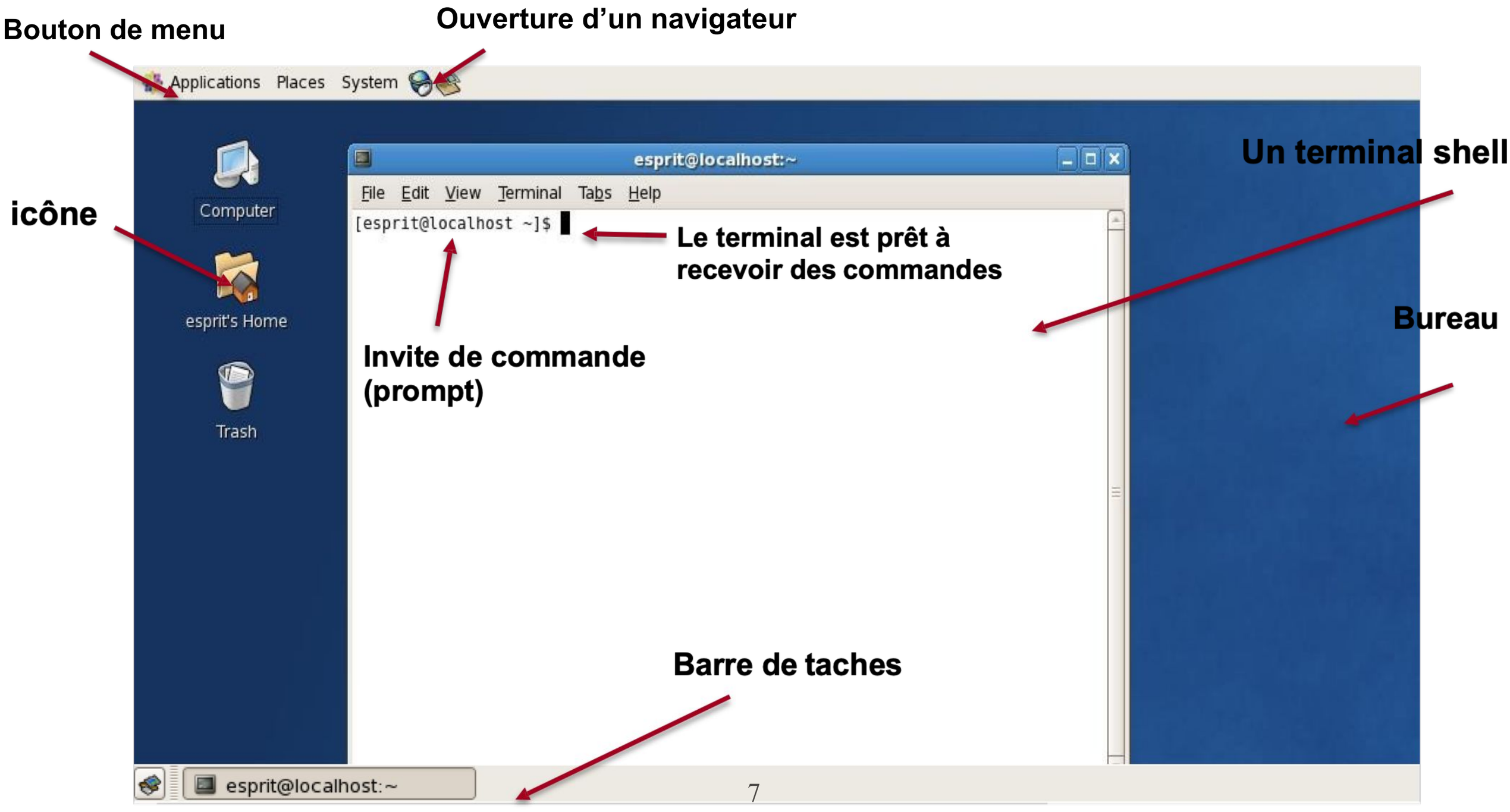
Première Connexion

- La première connexion d'un utilisateur Linux se fait grâce à un gestionnaire de connexion « login manager », tels que; xdm, gdm, kdm, etc.
- Un gestionnaire de connexion permet de lancer une session graphique.
- Une session graphique comporte essentiellement :
  - Un gestionnaire de fenêtre (window manager) gère la manipulation et l'apparence des fenêtres.
  - Un environnement de bureau (KDE, Gnome) peut être utilisé en plus, pour pouvoir utiliser les barres des tâches et les icônes sur le bureau.

Environnement Graphique

- À l'origine, les systèmes Unix n'étaient pas dotés d'une interface graphique, mais seulement d'une console texte.
- L'interface graphique est rajoutée sur un système Unix par le biais d'un serveur.
- Sous Linux, ce serveur s'appelle Xfree, aussi appelé serveur X.

Environnement Graphique



Environnement Graphique : Prompt

- Le prompt est structuré comme suit :

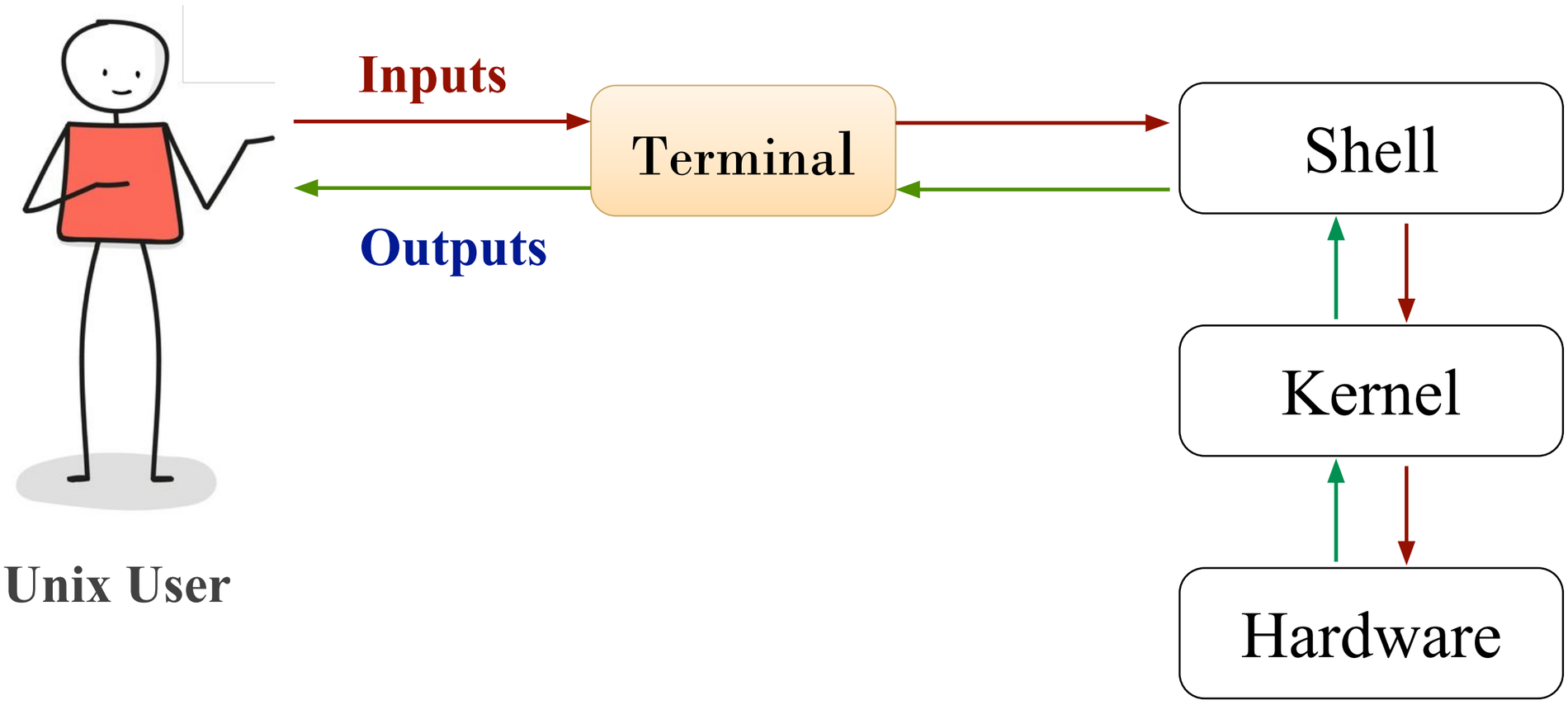


1. Le login ou le nom de l'utilisateur connecté
2. Nom de l'hôte ou de la machine
3. La position actuelle dans le système de fichier
4. Types d'utilisateurs ($: utilisateur simple / # : admin)

Terminal Vs Shell

- Terminal = Un environnement graphique d’entrée / sortie de texte qui permet à l'utilisateur d’introduire des commandes Shell.
- Shell = Un interpréteur de commande + Un langage de programmation.

Terminal Vs Shell

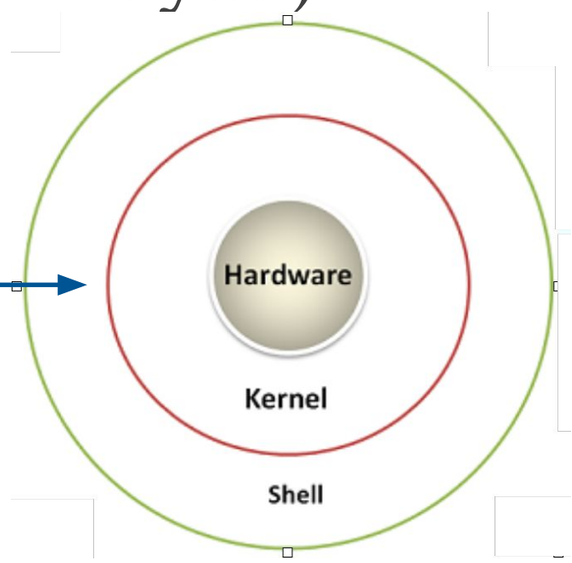


Shell : Définition

- Shell est une « c*oquille* » qui entoure le noyau Unix.
- Shell présente une interface entre l'utilisateur et le système d'exploitation.
- Ainsi, le shell est un programme qui permet à l'utilisateur de dialoguer avec le coeur du système (le noyau).



Coquille



Shell : Interpréteur de commandes

- Shell interprète des commandes, des programmes ou des fichiers exécutables.
- Il assure :
  - Affichage de l'invite de commande ou prompt ($) d'attente de lecture au clavier.
  - Lecture d'une commande (validée par Return ou Entrée).
  - Analyse syntaxique (découpage en mot).
  - Interprétation des caractères spéciaux.
  - Exécution de la commande et retour au début.

Shell : Langage de programmation

- Shell fournit à l'utilisateur un langage de programmation interprété.
- Il comporte :
  - Les notions de variable, d'opérateur arithmétique, de structure de contrôle, de fonction, des opérateurs spécifiques (|;).

Différents Shell & Fonctionnalités

- Il existe plusieurs types de Shells.

- Pour lister les shells supportés par votre machine, il suffit de taper la commande « **cat /etc/shells** ».
- Parmi les plus courants :
  - **Bourne Shell** (« /bin/sh ») : le Shell par défaut sous Linux
  - **Korn Shell** (« /bin/ksh ») : Deux versions majeurs sont aujourd'hui couramment utilisées (ksh 88 et ksh 93) ;
  - **cShell** (« /bin/csh ») pour les utilisateurs préférant un langage apparenté au « C » ;
  - **Job control Shell** (« /bin/jsh ») ;
  - **tcShell** amélioré (« /bin/tcsh ») améliorant le cShell tout en lui restant compatible ;

Différents Shell & Fonctionnalités

- Chaque utilisateur possède un shell par défaut, qui sera lancé à l'ouverture d'une invite de commande.
- Deux modes d'utilisation du Shell :
  - Interactif: l'utilisateur saisit et exécute ses lignes de commandes une par une dans un terminal ;
  - Non interactif: Shell interprète un ensemble de commandes à partir d'un fichier appelé *shell script*.

Commandes Shell

- Une ligne de commande est une chaîne de caractère constituée d'une commande ainsi que des arguments (paramètres) optionnels.
- Les commandes linux ont la syntaxe générale suivante :

Commande [Options] [Argument]

- Commande est le nom d'une commande interne ou un programme exécutable.
- Options : la plupart des commandes possèdent des options facultatives qui en modifient le comportement.
- Arguments/Paramètres sont les données d'entrées à la commande.

Commandes Shell

- Le shell distingue deux sortes de commandes : Commandes internes & Commandes externes.
  - Commande interne: Commande dont le code est implanté au sein de l'interpréteur de commande.
    - Exemples de commandes internes : cd , echo , pwd
- Commande externe: Commande dont le code se trouve dans un fichier.
  - La localisation du code d'une commande externe doit être connue du shell pour qu'il puisse exécuter cette commande.

Commandes Shell : Utilitaires d'aide

- Il existe une multitude de commandes sous linux. Il est difficile de se rappeler de l'intégralité de ces dernières.
- Un ensemble d'utilitaires d'aide ont été mis à la disposition des utilisateurs pour faciliter la manipulation de cet environnement.

Commandes Shell : Utilitaires d'aide

man [Commande]

- La commande « **man** » permet de fournir une page manuelle d'une commande envoyée en argument.

```
LS(1)                          User Commands                          LS(1)

NAME
       ls - list directory contents

SYNOPSIS
       ls [OPTION]... [FILE]...

DESCRIPTION
       List  information  about  the FILEs (the current directory by default).
       Sort entries alphabetically if none of -cftuvSUX nor --sort.

       Mandatory arguments to long options are  mandatory  for  short  options
       too.

       -a, --all
              do not ignore entries starting with .
```

Commandes Shell : Utilitaires d'aide

info [commande]

- La commande « **info** » permet de fournir une page info d'une commande envoyée en argument.

```
File: coreutils.info,  Node: cp invocation,  Next: dd invocation,  Up: Basic op\
erations

11.1 `cp': Copy files and directories
=====================================

`cp' copies files (or, optionally, directories).  The copy is
completely independent of the original.  You can either copy one file to
another, or copy arbitrarily many files to a destination directory.
Synopses:

     cp [OPTION]... [-T] SOURCE DEST
     cp [OPTION]... SOURCE... DIRECTORY
     cp [OPTION]... -t DIRECTORY SOURCE...

   * If two file names are given, `cp' copies the first file to the
     second.
```

Commandes Shell : Utilitaires d'aide

apropos [Commande]

- La commande « **apropos** » est une commande POSIX qui permet de lister les manuels dont la description comprend les mots passés en arguments.

```
[esprit@localhost ~]$ apropos copy
bcopy                  (3)  - copy byte sequence
bcopy                  (3p)  - memory operations (LEGACY)
bcopy [bstring]        (3)  - byte string operations
copysign               (3)  - copy sign of a number
copysign               (3p)  - number manipulation function
copysignf [copysign] (3)  - copy sign of a number
copysignf [copysign] (3p)  - number manipulation function
copysignl [copysign] (3)  - copy sign of a number
copysignl [copysign] (3p)  - number manipulation function
cp                     (1)  - copy files and directories
cp                     (1p)  - copy files
```

Commandes Shell : Utilitaires divers

- date : affiche la date,
- cal : affiche le calendrier,
- who : affiche les utilisateurs connectés,
- whoami : affiche votre nom d'utilisateur,
- uname -a : affiche toutes les informations sur le système,
- whatis : affiche une description de la commande,
- hostname: afficher le nom de la machine,
- halt ou shutdown -h now ou init 0: éteindre l'ordinateur,
- reboot ou shutdown -r now ou init 6: rebooter l'ordinateur,

Commandes Shell d'identification

type [Commande]

- La commande « **type** » permet de retourner le chemin absolu d'une commande externe ou d'indiquer qu'il s'agit d'une commande interne (shell builtin).

which [Commande]

- La commande « **which** » recherche les exécutables des commandes selon les chemins indiqués au niveau de la variable PATH. Elle retourne le chemin absolu de la commande externe. S'il s'agit d'une commande interne, elle

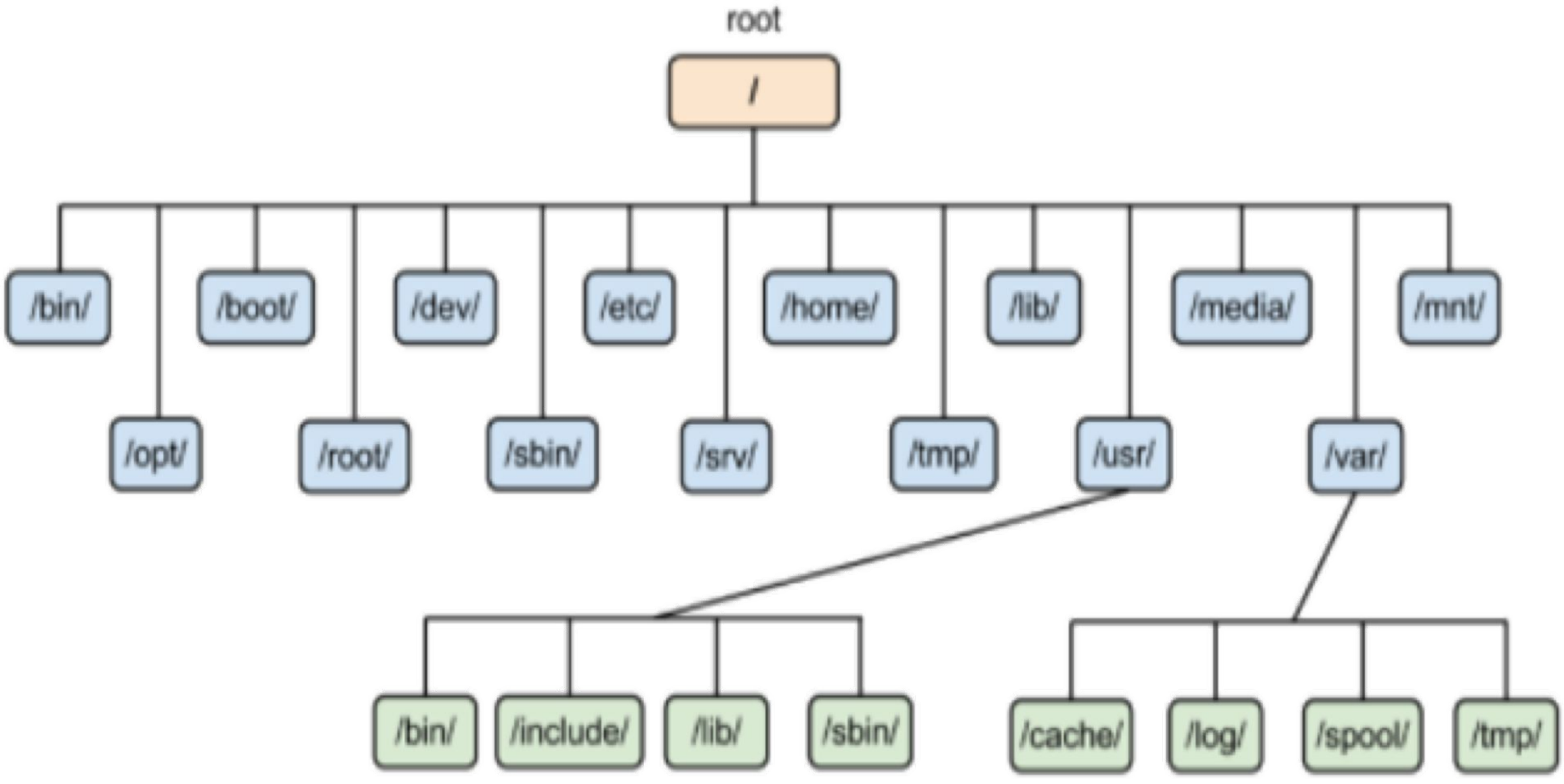
Commandes Shell d'identification

whereis [Commande]

- La commande « **whereis** » a le même fonctionnement que la commande « which ». En outre, elle affiche le chemin vers les pages manuelles des commandes.

Manipulation des Fichiers & Répertoires : FHS

FHS: Filesystem Hierarchy Standard



Manipulation des Fichiers & Répertoires : FHS

- / : est le répertoire racine par analogie aux racines d'un arbre, représentant l'arborescence des répertoires et fichiers qui va se développer à partir de ce point. C'est donc le point de départ de toute la hiérarchie du système de fichiers.
- /bin : est le répertoire contenant les commandes de base accessibles par défaut à tous les utilisateurs, bin pour binary, c'est à dire fichier exécutable.

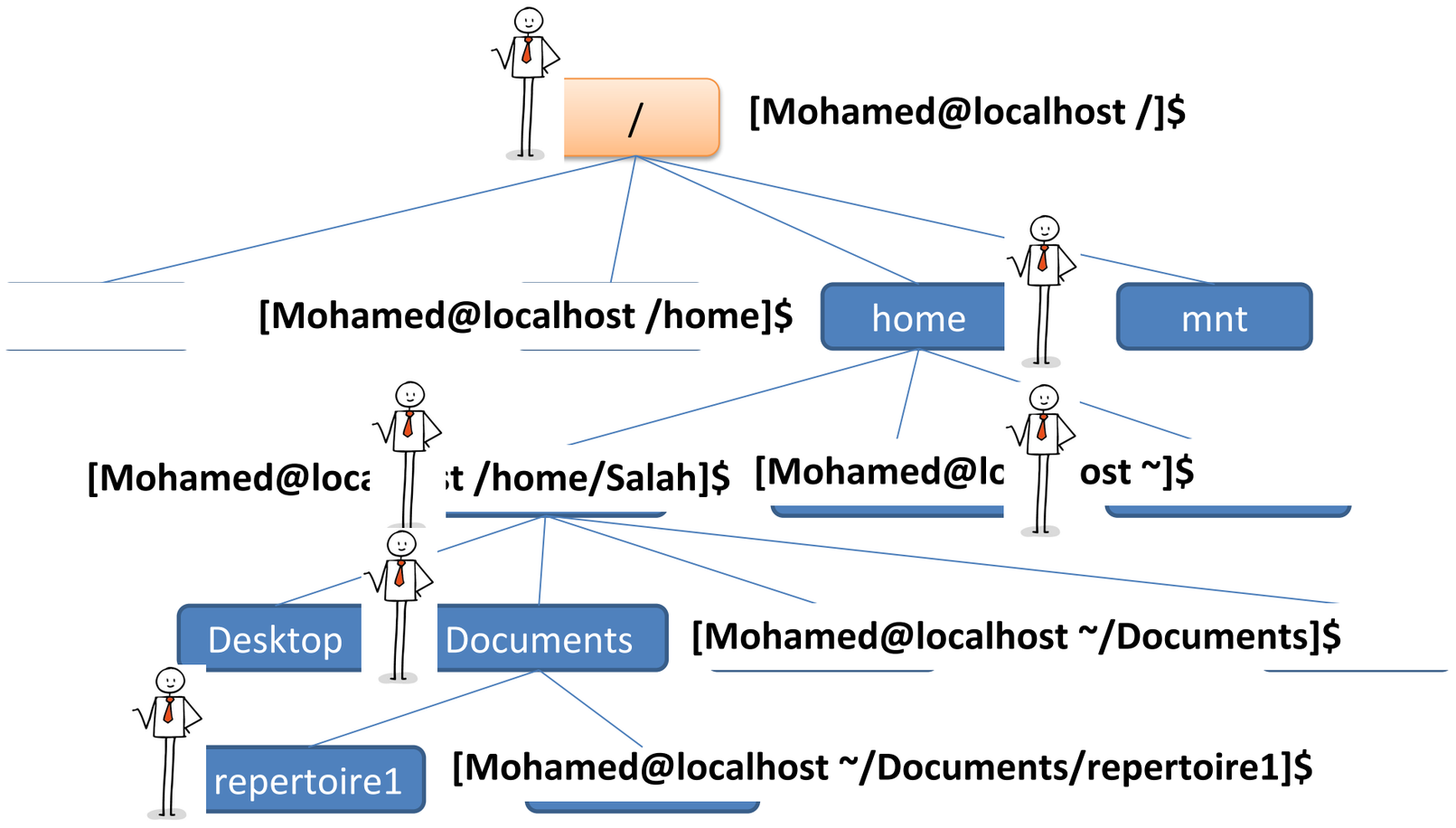
Manipulation des Fichiers & Répertoires : FHS

- /boot est un fichier statique du chargeur d'amorçage. Il contient les fichiers nécessaires au démarrage du système (notamment Grub)
- /dev désigne *"device"* ou périphérique. Ce dossier contient des fichiers spéciaux qui assurent la communication entre le système et les périphériques
- /etc présente des répertoires de fichiers de configuration au format textuel de plusieurs programmes et services sous Unix.

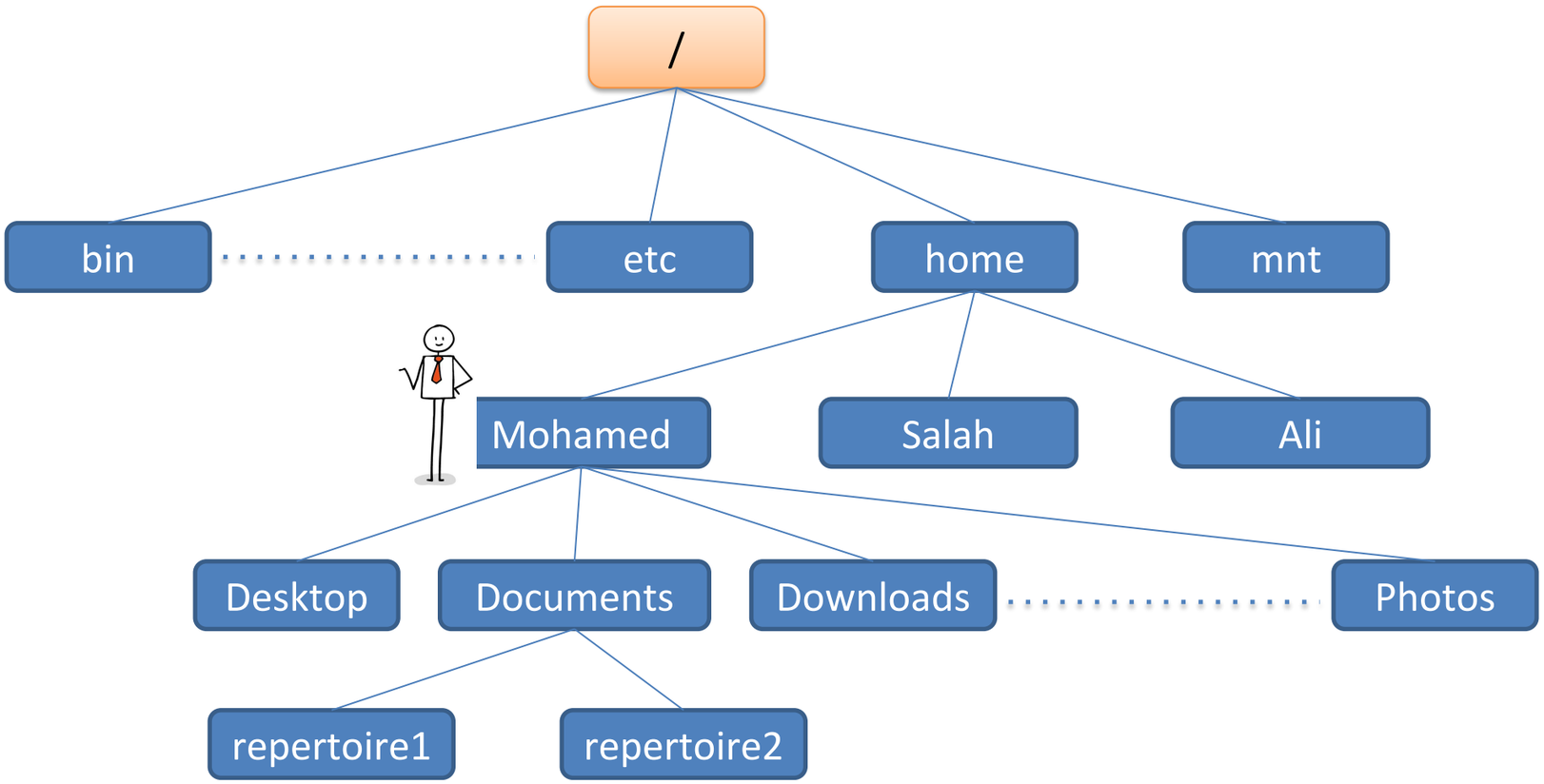
Manipulation des Fichiers & Répertoires : FHS

/bin	Les commandes système
/boot	Les fichiers de démarrage du noyau (Grub, Lilo)
/dev	Les fichiers utilisés par les périphériques
/etc	Les fichiers de configuration locale
/home	Contient tous les répertoires des utilisateurs
/lib	Des bibliothèques partagées entre les fonctionnalités du système
/mnt	Points de montages de périphériques temporaires
/sbin	Commandes spécifiques pour gérer le système (exécutables pour les admin)
/tmp	Fichier temporaires - généralement nettoyé au démarrage
/usr	Données partagées, en lecture seule (***UNIX System Resources***)
/var	Fichiers en cours d'utilisation et variables en contenu

Manipulation des Fichiers & Répertoires : FHS



Manipulation des Fichiers & Répertoires : FHS

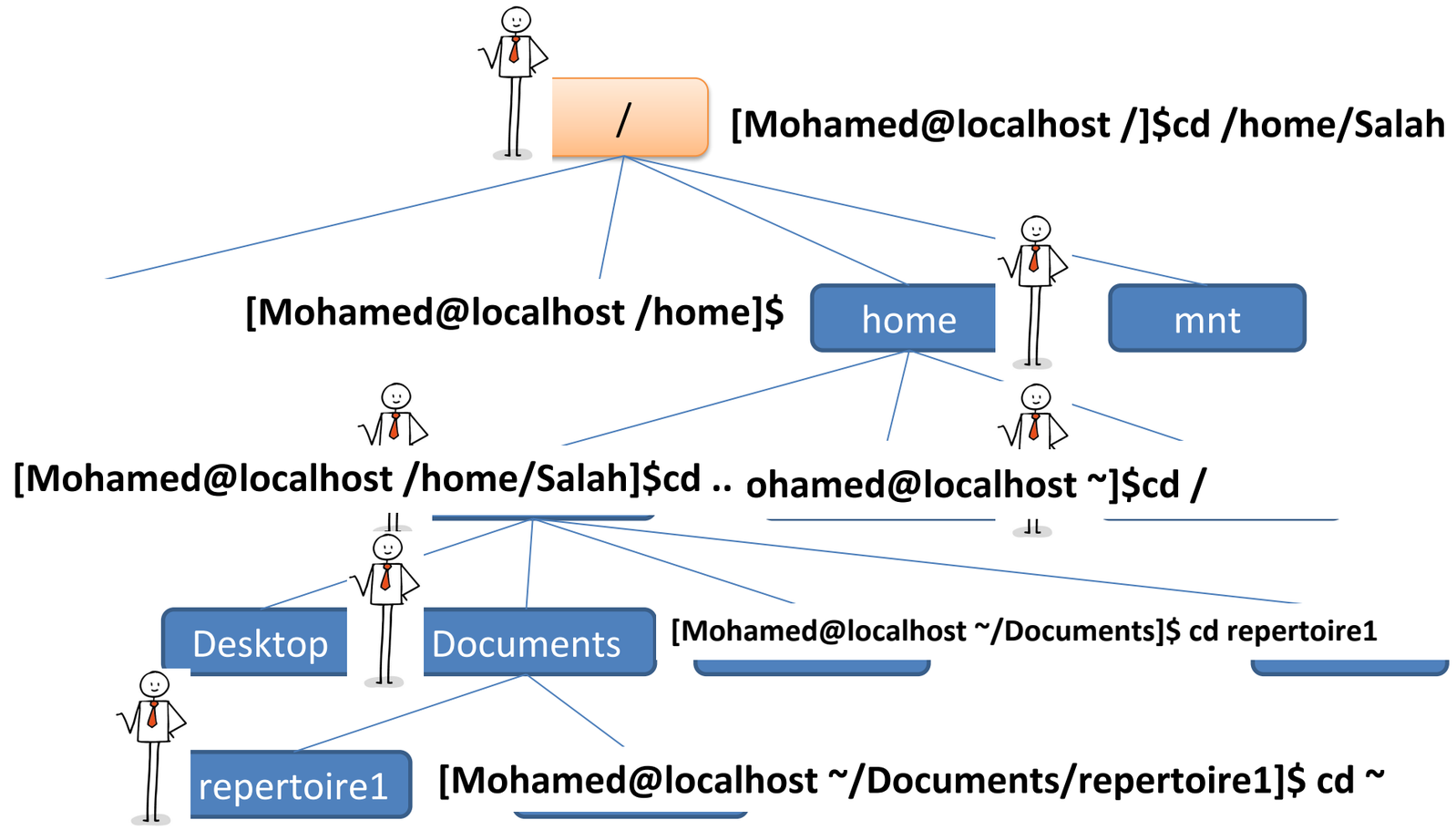


Commandes Shell : Manipulation des fichiers et des répertoires

Commande cd

- La commande «**cd** » permet de se déplacer dans l'arborescence. Sans paramètre, elle nous renvoie vers le répertoire personnel de l'utilisateur actuel, sinon au chemin spécifié en argument,
- Avec les arguments listés ci-dessous, elle permet de :
  - . : se déplacer dans le répertoire courant
  - .. : se déplacer vers le répertoire parent
  - - : se déplacer vers le répertoire précédent
  - / : se déplacer vers le répertoire racine

Manipulation des Fichiers & Répertoires



Commandes Shell : Manipulation des fichiers et des répertoires

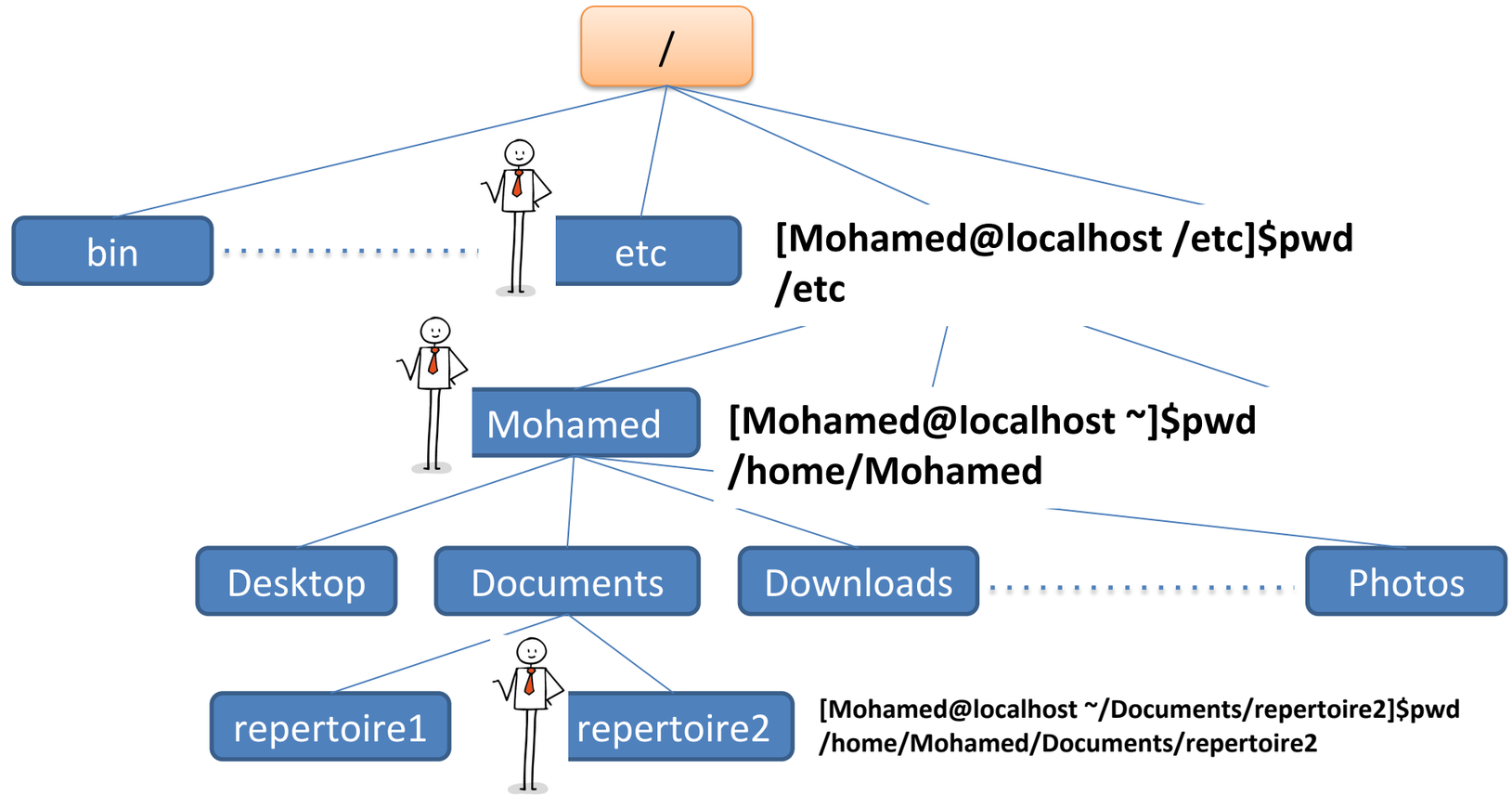
Commande pwd

- La commande « pwd » permet d'afficher le chemin absolu du répertoire courant.

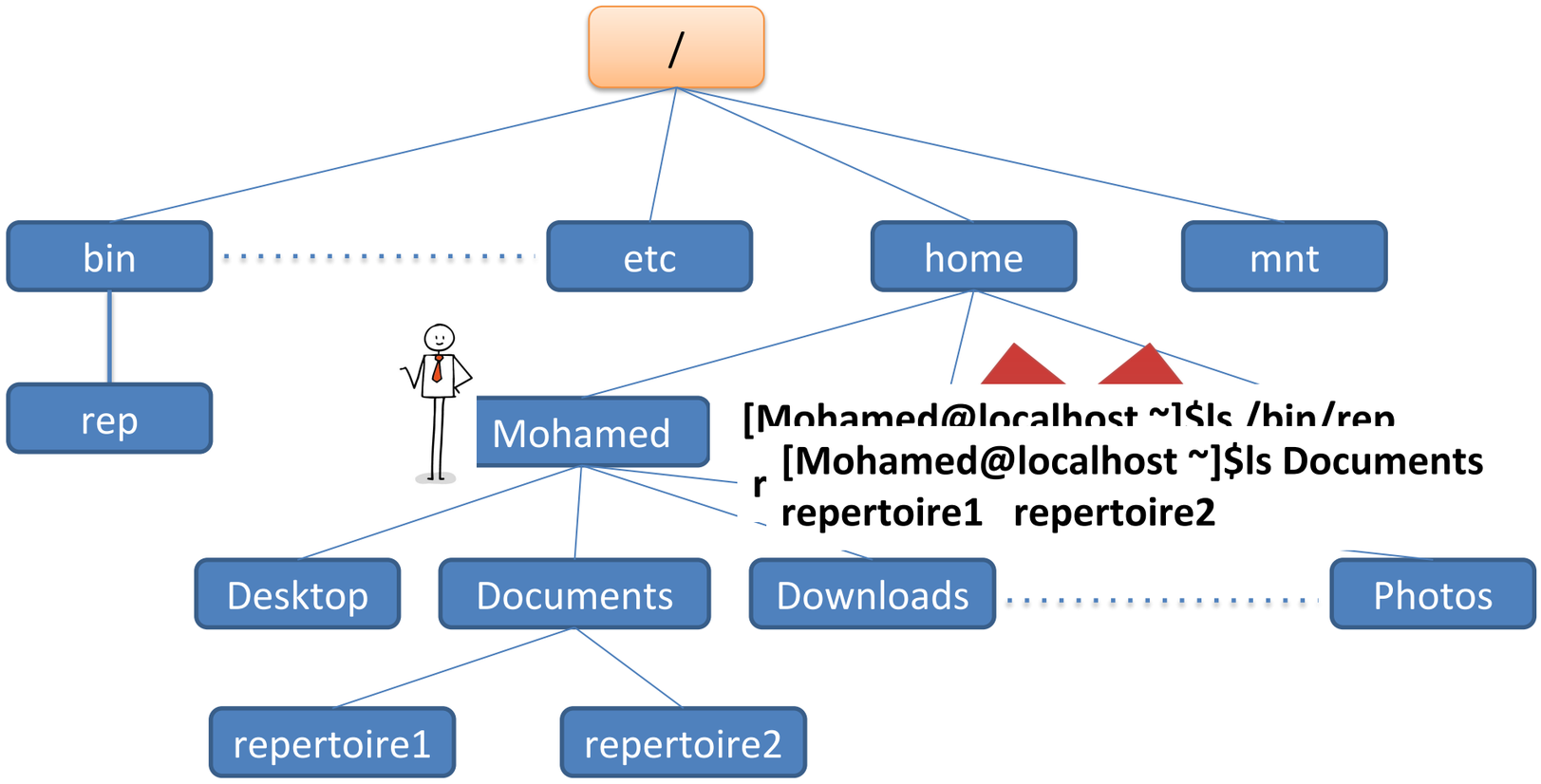
Commande ls

- Cette commande permet principalement de lister le contenu d'un répertoire. Nous pouvons lui associer plusieurs options dont les plus évidentes sont :
  - -l : Lister les métadonnées relatives à un fichier ou un répertoire.
  - -a : Afficher tous les fichiers, y compris les fichiers cachés.
  - -R : Pour assurer un affichage récursif (le contenu du

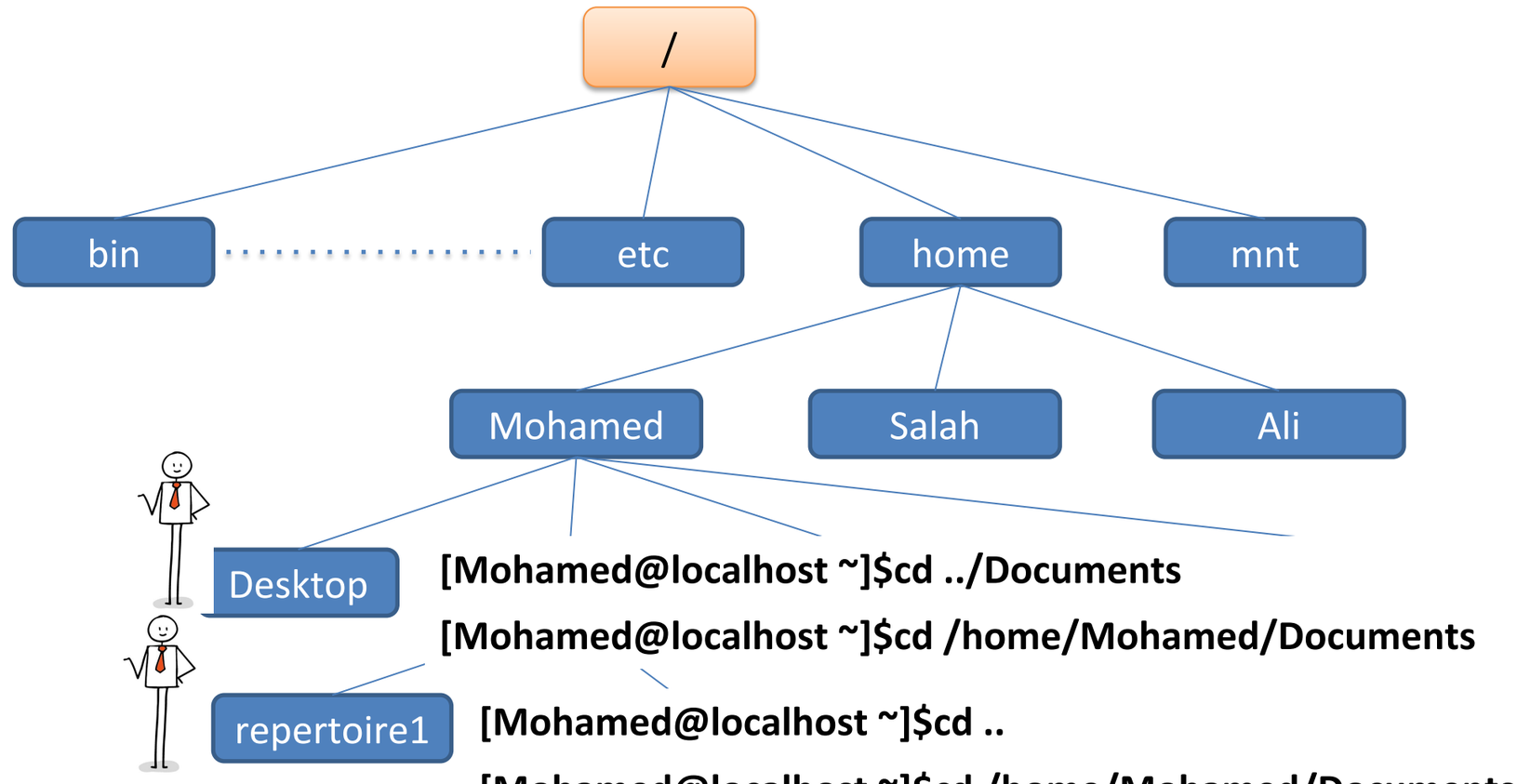
Manipulation des Fichiers & Répertoires



Manipulation des Fichiers & Répertoires



Manipulation des Fichiers & Répertoires : Chemin relatif vs Chemin absolu



Commandes Shell : Manipulation des fichiers et des répertoires

Commande touch

- La commande « **touch** » permet de créer un ou plusieurs fichiers vides.

Commande nano

- La commande « **nano** » est une commande d'édition de fichier.

Commande vi

- La commande « **vi** » est une commande d'édition de fichier.

Commande gedit

- La commande « **gedit** » est une commande d'édition de fichier.

Commande cat

- La commande « **cat** » permet de:

cat *fichier* Afficher le contenu d'un fichier

Commandes Shell : Manipulation des fichiers et des répertoires

Commande head

- La commande « **head** » permet d'afficher par défaut les dix premières lignes.
  - head -n nbre *fichier*: afficher les nbre premières lignes du fichier
  - head -c nbre *fichier*: afficher les nbre premiers caractères du fichier.

Commande tail

- La commande « **tail** » permet d'afficher par défaut les dix dernières lignes.
  - tail -n nbre *fichier*: afficher les nbre dernières lignes du fichier

Commandes Shell : Manipulation des fichiers et des répertoires

Commande wc

- La commande « **wc** » permet d'afficher le nombre de lignes, nombre des mots, nombre d'octets.
  - ***wc -l*** *fichier*: Afficher le nombre de lignes du *fichier*.
  - **wc -w** *fichier*: Afficher le nombre de mots du *fichier*.
  - **wc -c** *fichier*: Afficher le nombre d'octets du fichier.
  - **wc -m** *fichier*: Afficher le nombre de caractères du fichier.

Commande split

- La commande « **split**» permet de fragmenter un fichier. Chaque fragment est stocké dans des fichiers PREFIXaa, PREFIXab …
  - *split -b* size *fichier partie*: Fragmente *fichier* en des fichiers de taille size (octets) nommés partieaa, partieab, etc.

Commandes Shell : Manipulation des fichiers et des répertoires

Commande paste

- La commande « **paste**» permet de joindre des fichiers horizontalement .
  - ***paste –d*** « ***délimiteur***» *fichier*1 *fichier*2 : concatène horizontalement chaque ligne du *fichier*1 avec la ligne correspondante du *fichier2* séparé par « ***délimiteur*** » .
  - ***paste –s*** *fichier1 fichier2* : concatène verticalement les lignes du fichier 1 avec les lignes du fichier 2 .

Commande join

- La commande « **join** » permet du fusionner les lignes de deux fichiers ayant des champs communs.

Commandes Shell : Manipulation des fichiers et des répertoires

Commande mkdir

- La commande « mkdir » permet la création d'un répertoire sous le chemin spécifié en paramètres. Pour la création d'une arborescence, il faut ajouter l'option « **-p** ».

Commande cp

- La commande « **cp** » permet de copier une entité (fichier/répertoire) d'un emplacement à un autre.
  - -i: interactif: demande l'autorisation d'écraser.
  - -r: récursif: Copie les sous-répertoires et leur contenu.

Commande mv

Commandes Shell : Manipulation des fichiers et des répertoires

Commande rmdir

- La commande « rmdir » permet de supprimer un répertoire vide.

Commande rm

- Cette commande permet de supprimer une entité (fichier/répertoire).
  - -i: interactif: demande l'autorisation d'écraser.
  - -r: récursif: supprime les sous-répertoires et leur contenu.

Variables d'environnement

```
[esprit@localhost ~]$ esprit
bash: esprit: command not found
[esprit@localhost ~]$ man esprit
No manual entry for esprit
[esprit@localhost ~]$ apropos esprit
esprit: nothing appropriate
```

Commande introuvable?????
Pourquoi le Shell n'arrive pas à trouver cette commande

Variables d'environnement

- Une variable d’environnement est une valeur dynamique chargée en mémoire, elle est accessible par plusieurs programmes du système d’exploitation.
- Ces variables sont chargées d’une façon automatique lors de démarrage.

Variables d'environnement

- *PATH* : indique la liste des chemins où trouver les programmes à lancer.
- *SHELL* : indique l'interpréteur shell utilisé par défaut.
- *HOME* : contient le chemin absolu vers le répertoire personnel de l'utilisateur connecté.
- *LOGNAME* : pour le nom de l'utilisateur
- *HISTFILE* : pour le fichier historique.
- *HISTSIZE* : pour la limite de commandes historiques accessibles.

Variables d'environnement

- Pour afficher la valeur d'une variable:

echo $VAR_ENV

- Pour déclarer des variables d'environnement :

VARNAME=valeur

export VARNAME

 Export rend la variable visible pour tous les shells et les programmes

- Pour remplacer les deux commandes précédentes:

export VARNAME=valeur

- Pour effacer le contenu d'une variable: ***unset VARNAME***
- Pour afficher toutes les variables d'environnement: ***env***

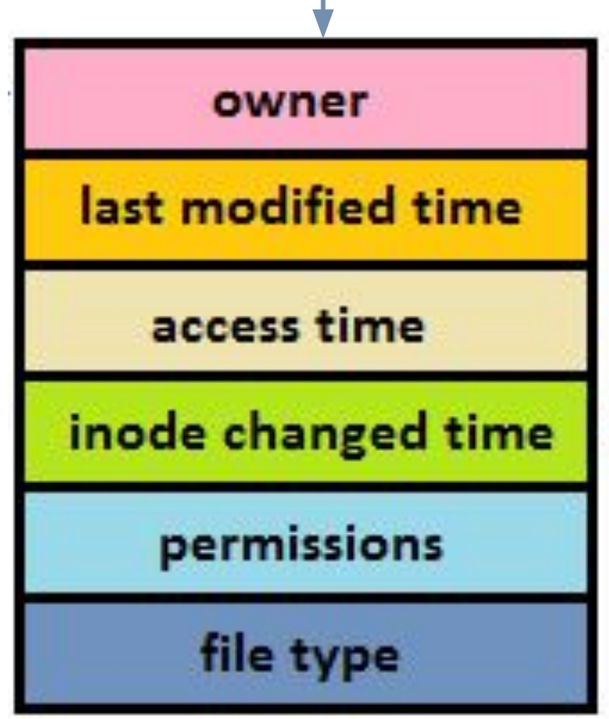
Inode : «Index » Node

- Les inodes (contraction de « index » et « node », en français : nœud d'index) sont des structures de données contenant des informations concernant les fichiers stockés dans certains systèmes de fichiers (notamment de type Linux/Unix).
- À chaque fichier correspond un numéro d'inode (i-number) dans le système de fichiers dans lequel il réside, unique au périphérique sur lequel il est situé.
- Les inodes sont créés lors de la création du système de fichiers.
- La quantité d'inodes (généralement déterminée lors du formatage et dépendant de la taille de la partition) indique le nombre maximum de fichiers que le système de fichiers peut contenir.

Inode : «Index » Node

Directory

Name	Inode Number (01245)



Inode (01245)
Metadata

Lien Physique Vs Lien Symbolique

- Un lien est un fichier spécial qui permet de faire plusieurs références à un même fichier sur le disque dur. L'intérêt est de pouvoir accéder à un fichier de plusieurs endroits de l'arborescence. Il existe deux types de liens : les liens physiques et les liens symboliques.

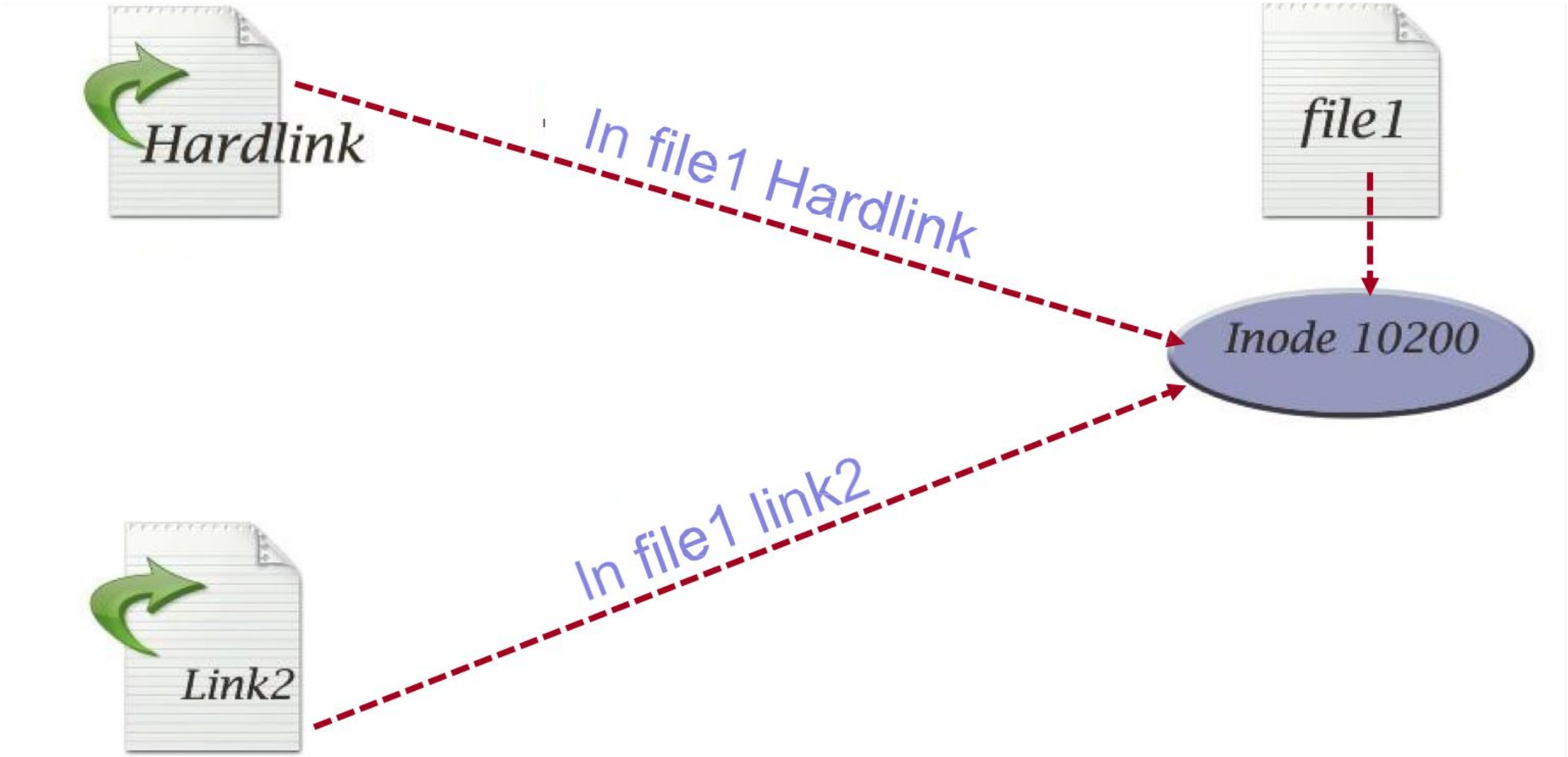
Lien Physique Vs Lien Symbolique

lien physique

- Le système de fichier de Linux enregistre les fichiers sur la base d'un numéro et pas sur la base d'un nom. Grâce à un lien physique, un fichier peut avoir plusieurs noms/chemin d'accès, à un même fichier en pointant sur un numéro unique d'**inode** qui identifie le fichier dans le système de fichiers.

Lien Physique Vs Lien Symbolique

lien physique



Lien Physique Vs Lien Symbolique

lien physique

- La commande utilisée pour créer un hard link est la commande **ln**

```
[esprit@localhost ~]$ ln file1 hardlink
[esprit@localhost ~]$ ln file1 link2
[esprit@localhost ~]$ ls -li file1 hardlink link2
9910 -rw-rw-r--. 3 esprit esprit 1 Sep  4 09:50 file1
9910 -rw-rw-r--. 3 esprit esprit 1 Sep  4 09:50 hardlink
9910 -rw-rw-r--. 3 esprit esprit 1 Sep  4 09:50 link2
```

Le numéro affiché en utilisant la commande ls –i est le numéro d'inode.

- On remarque que les liens physiques créés pointent sur le même inode.

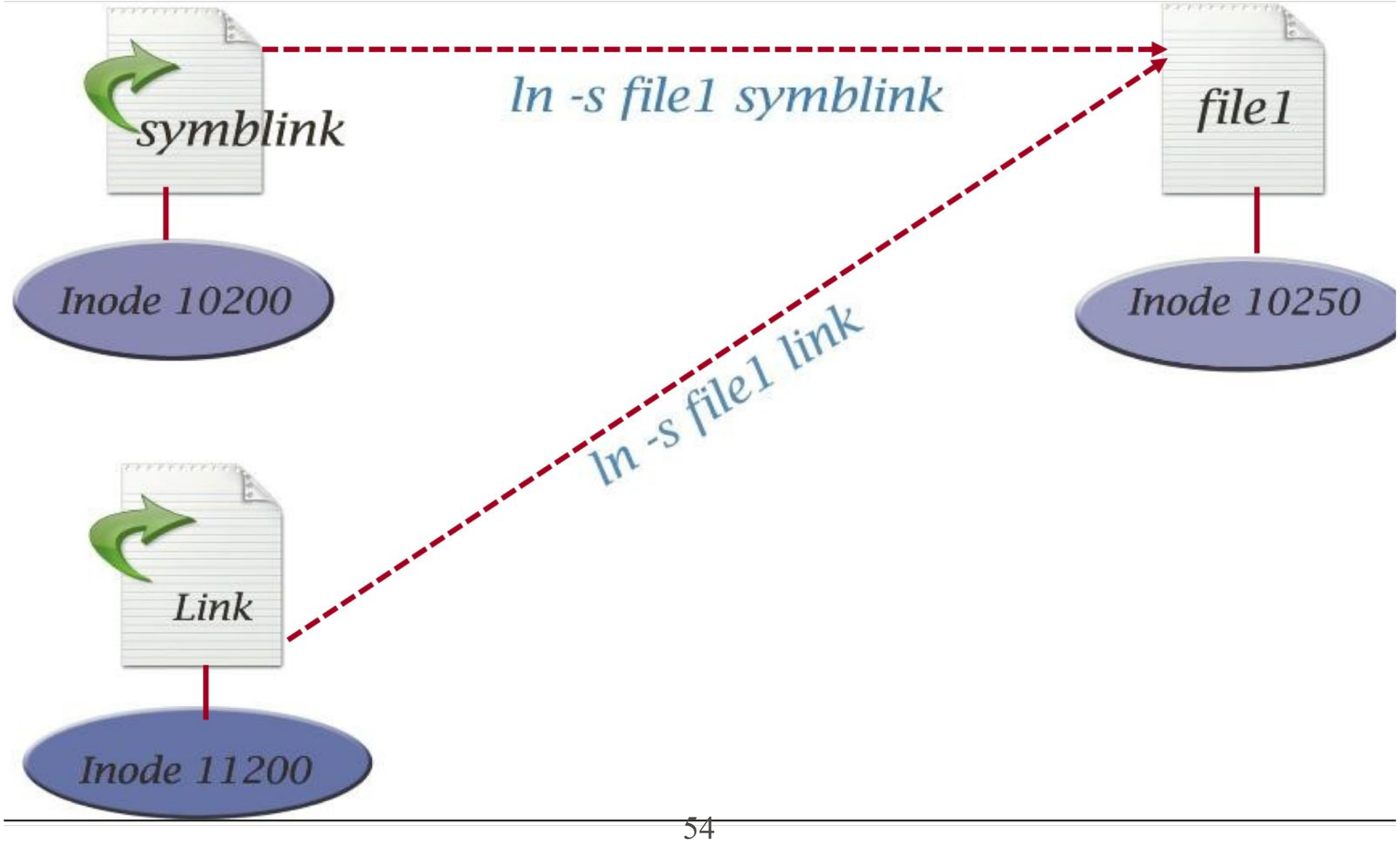
Lien Physique Vs Lien Symbolique

lien symbolique

- Les liens symboliques ressemblent plus aux « raccourcis» sous Windows. La plupart du temps, on crée des liens symboliques sous Linux pour faire un raccourci, et non des liens physiques.
- Le principe du lien symbolique est que l'on crée un lien vers un autre nom de fichier. Cette fois, on pointe vers le nom de fichier et non vers l'inode directement

Lien Physique Vs Lien Symbolique

liens symboliques



Lien Physique Vs Lien Symbolique

lien symbolique

- La commande utilisée pour créer un lien symbolique est la commande **ln avec l'option -s**.

```
[esprit@localhost ~]$ ln -s file1 symblink
[esprit@localhost ~]$ ln -s file1 link
[esprit@localhost ~]$ ls -li file1 symblink link
 9910 -rw-rw-r--. 3 esprit esprit 1 Sep  4 09:50 file1
10560 lrwxrwxrwx. 1 esprit esprit 5 Sep  5 04:22 link -> file1
10537 lrwxrwxrwx. 1 esprit esprit 5 Sep  5 04:22 symblink -> file1
```

Le numéro affiché en utilisant la commande ls –i est le numéro d'inode.

- On remarque que les liens symboliques créés pointent sur des inodes différentes.

Lien Physique Vs Lien Symbolique

- Un lien physique utilisant le même numéro d'inode, il se limite à la même partition.
- Pour traverser les limites des partitions, Unix introduit des liens dits symboliques.
- Un lien physique pointe sur la même inode du fichier contrairement au lien symbolique.
- Un lien physique ne peut être créé que pour des fichiers.
- Un lien symbolique peut être créé pour un fichier comme pour un répertoire.

Références

- *[1] : Préparation à la certification LPIC1 : Linux - Sébastien Rohaut.*
- *[2] : 2017 Initiation Unix France – UNIX & Linux, 3e édition – David Rossignol.*